PROJECT REPORT

REAL Vs FAKE, AND VIDEO DETECTION

Submitted for

CSET419 – Introduction to Generative AI

Submitted by:

(<E23CSEU2326>) NAME- KRITIKA

(<E23CSEU2316>) NAME- KHUSHI

Submitted to

Prashant Kumar

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

Candidate's Declaration

I hereby declare that the work presented in this report entitled "**REAL Vs FAKE, AND VIDEO DETECTION**" in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering** submitted in the department of Computer Science & Engineering and Information Technology, Bennett University of Information Technology Greater Noida is authentic record of my own work carried out over a period from July 2025 to December 2025 under the supervision of **Prashant Kumar.** The matter embodied in the report has not been submitted for the award of any other degree or diploma.

Kritika

E23CSEU2326

Khushi

E23CSEU2316

This is to certify that the above statement made by the candidate is true to the best of my knowledge.

Prashant Kumar

Computer Science & Engineering

Dated: 16-04-2026

ACKNOWLEDGEMENT

I would like to thank and express our gratitude to our Project supervisor **Prashant Kumar** for the opportunity that he provided us with this

wonderful project “Real Vs Fake, and Video Detection”. The outcome would not

be possible without his guidance. This project taught me many new things and

helped to strengthen concepts of **stanford CS231n (2024)** study on Deep Learning for Deepfake Detection. Next, I would like to

express my special thanks to the Lab Assistant for cordially contacting us and

helping us with finishing this project within the specified time. Lastly, I would

I like to thank my friends and parents for their help and support.

Kritika

E23CSEU2326

Khushi

E23CSEU2316

TABLE OF CONTENTS

S.No.	TITLE	Page No.
1.	ABSTRACT	5
2.	INTRODUCTION	6
3.	PROBLEM STATEMENT	7
5.	LITERATURE REVIEW & RESEARCH FOUNDATION	8-16
6.	Hardware/Software Required	17
7.	Conclusion	18-20
8.	Future Scope	21-22
9.	GitHub link for complete project	23

ABSTRACT

Something big happened when machines started making fake pictures and videos. Instead of just creating new things, they began tricking people about what's real. Our work dives into that messy space where artificial intelligence meets truth-finding tools. Backed by ideas from Stanford's CS231n course in 2024, we built something that spots phony visuals - especially deepfakes made with GANs or VAEs. It doesn't rely on one method alone; layers do different jobs: EfficientNet-B4 catches odd visual glitches while LSTMs watch how movement flows across time. Surprisingly steady results came through testing, showing it can tell human faces apart from machine-made ones pretty well. What looks like small digital hiccups often gives the fakes away - if you know where to look.

Introduction

Out of thin air, algorithms now generate fresh material - sound, pictures, even moving scenes. Think DALL-E painting visuals or Midjourney shaping fantasy landscapes. Yet alongside art comes risk: false clips spread fast when tools mimic reality too well. Sora crafts video from words, blurring fact and fiction without warning. Creativity gains power, though truth often stumbles in the rush.

One way to spot deepfakes? Look where things don't quite add up. Instead of making fake images, like generative models do, we focus on catching them. A close look often reveals odd details most people miss. Think about light falling wrong across a face. Or eyes that blink too little, or far too much. Even how pixels line up can betray the machine behind them. Those grid-like traces come from certain steps inside GANs. Behind every synthetic image, tiny flaws stay hidden in plain sight.

Problem Statement

Fast changes in artificial intelligence that makes things look real have shifted how we see online content. Because tools like GANs, VAEs, or diffusion systems exist, people can now craft detailed fake visuals. These creations blur what is genuine, making it tough to tell truth from fiction. Though useful for art or design, such power also brings risk when misused. Seeing something clear does not mean it actually happened. With deepfakes spreading fast, trust in images fades quickly. What appears solid might be built on invisible code instead of fact.

The core challenges addressed by this research project are:

Now picture this - fake videos that look real, identities stolen by machines, lies built one pixel at a time. When anyone can twist truth with just a few clicks, proof starts losing its power. Journalists dig for facts but hit walls of illusion instead. People once believed what they saw online; now doubt creeps in fast. Digital sleuthing stumbles when every photo might be lying. Safety crumbles not from force, but from confusion.

Out of nowhere, tiny details in artificial faces feel too real. Sometimes light hits just right on synthetic skin, blurring what's actual. Eyes struggle to tell apart the made-up from living flesh. Little by little, trust in seeing vanishes. What looks natural might be built in code. Reality slips when imitation wins silently.

Even if single frames look right, moving clips made by AI can wobble in odd ways. Objects might twitch between seconds because positions shift without logic. A nose could drift sideways then snap back. Most tools designed to catch fakes miss these hiccups entirely. Smoothness breaks down when moments do not line up properly.

Some tools spot tiny flaws machines leave behind. These marks - like grid-like patterns from image resizing - show where tech has been used. One common trace comes from how pixels are stretched in certain models. Not every flaw looks the same, but some repeat often enough to be telling. Spotting them means watching for small repeating structures most people miss. Systems able to catch these traces could help tell real from synthetic. The trick lies in training software to notice what eyes usually skip. What matters is consistency across different outputs and methods. Hidden signals exist if you know where to look. Machines might create clean results on surface - but close inspection tells another story..

LITERATURE REVIEW & RESEARCH FOUNDATION (Stanford CS231n)

- Methodology based on Stanford CS231n 2024 deep learning for deepfake detection
- Fingerprints come out of every generator - ProGAN, StyleGAN, or StarGAN - each shaped by its own design quirks. These patterns live in the fine details, sharp and repeating. What we built learns to spot them, not guessing, just seeing what's etched into each image. The signals hide in plain sight, clear once you know where to look.
- One moment might show a face just off from the next. Frames come out scattered, not linked. A character could appear different in two seconds flat. Backgrounds shift when they should stay put. Each piece gets made alone, causing little mismatches. Things move oddly even if nothing changed. The scene feels shaky without meaning to.
- One way they improve separation between real and fake data points is by using Binary Cross-Entropy Loss. Another tool helps shape how these groups form - that method is called Triplet Loss. Together, through their joint effect, the model pushes similar examples closer while pulling different ones apart. This happens inside a space where each point has many

features. What stands out is how the two losses work side by side, sharpening the divide. Clusters become more distinct because of this pairing. The result? A clearer split between what's genuine and what's not..

2.3 Purpose and Objectives

- A fresh look at movement and position comes from splitting tasks into two paths. One handles where things are, while the other tracks how they change over time. This setup runs both sides at once, balancing location details with motion flow. Accuracy grows because space and timing get separate focus. The method builds precision by treating each aspect in its own lane
- 4.1 Data Acquisition and Preprocessing
- Handling what you put in: works with MP4 files, also handles AVI, takes JPEG too. Format flexibility means video clips or still images go through without trouble.
- Starting with MTCNN, we pull out faces using a method built on layered convolutional networks. Since fake changes often cluster around eyes, mouth, and skin, locking onto the face matters. What drives this step is how heavily manipulated areas tend to gather right there.
- Every fifth frame gets picked to help track changes over time without slowing things down.
- 4.2 Spatial Features Extracted Through CNN
- ImageNet helped train the model known as EfficientNet-B4.
- Looking closely, it checks for oddities in space - skin that feels off, earrings that don’t match, eyes reflecting light wrong - things often found in fake images.
- 4.3 Modeling Time Sequences with RNN Streams
- Model: Bi-directional LSTM.

- Starting with the CNN's output, data flows into the LSTM. Moving through time, the LSTM checks if motions - like smiling or blinking - follow natural movement rules. When odd jumps or flashes show up on the face, they get marked as fake. What stands out is unnatural timing - it raises red flags..

HARDWARE AND SOFTWARE REQUIREMENT

Computers study information, including video, so they can spot when something has been faked. Learning happens by watching patterns, then recognizing what looks off. When images move strangely or sounds mismatch, the system flags them. It digs into details most people skip. False content gets caught because it often stumbles on small flaws. Machines notice these hiccups faster than eyes do. What seems real at first might unravel under digital inspection. The goal sits quietly beneath: find truth hidden in pixels and frames.

A solid setup starts with a machine that can run the necessary programs. What matters most is having tools built for the task at hand. Without the correct apps, progress slows down fast. Running outdated systems blocks smooth performance. The whole thing depends on matching hardware to software needs. Smooth function follows only when both pieces fit just right.

Software Requirements

For the operating system, pick one - either a 64-bit version of Windows 10 or 11, otherwise go with Ubuntu 20.04 or 22.04. Though options differ, each must be 64-bit to work right. One fits if it's on that list, nothing else counts. Not every setup qualifies; only these belong.

Most people pick Ubuntu since it handles the graphics card more smoothly.

Python 3.10 or higher used.

What makes this one stand out is how well it runs alongside certain builds of PyTorch, also keeping pace with specific TensorFlow releases.

Deep Learning Frameworks:

Our computer model gets built using either PyTorch 2.1 or TensorFlow 2.15. Running it happens right after, powered by one of those tools instead of both at

once. Each choice works fine on its own, no extra layers needed. The whole process stays smooth without swapping parts mid-way.

Building the network becomes simpler thanks to Keras handling the finer points up front. It takes care of complex setup behind the scenes without extra steps needed.

Computer Vision Libraries:

When handling video feeds, OpenCV helps pull out individual frames while shifting between color formats. Though built for visuals, its strength shows most when breaking down motion into stills. Sometimes it adjusts hues smoothly others handle flow without lag. From stream to snapshot, the tool shifts how colors are seen across steps.

Finding faces in videos gets easier when tools like Dlib step in. Accurate alignment shows up through methods such as MTCNN. Precision comes alive once these systems process each frame. Clear positioning follows after detection takes place.

Data Science Libraries:

When working with numbers or organizing information, NumPy plus Pandas often come into play. Though separate tools, they handle tasks like calculations and sorting details smoothly together.

Pictures showing how the model learns come alive through Matplotlib alongside Seaborn. During training, progress takes shape on screen without extra fuss.

Web Framework:

Start building a site with Streamlit or go the Flask route instead. Uploads come through when people bring in media files. Results show right after the system processes them. A live display answers each submission.

Version Control:

Git runs the version control while GitHub hosts the code. Working together means changes stay tracked across machines.

Hardware Requirements

Processor (CPU):

A solid choice would be an Intel Core i7, 11th generation at least - or something more recent. Ryzen 7 chips from AMD fit the bill just as well. One could go with either option without losing performance ground. Modern tasks demand that level of computing muscle, nothing less. Either brand holds up under heavy workloads when built right.

Faster frame prep happens when chips split tasks across cores.

Graphics Processing Unit

A solid choice would be something such as an NVIDIA GeForce RTX 3060, though stronger models work too - just make sure it packs a minimum of 8GB in dedicated graphics memory.

Calculations move faster because of the GPU's CUDA cores.

Random Access Memory

A solid setup starts at 16GB of RAM - though stepping up to 32GB makes things smoother. What really matters? Having enough headroom when juggling tasks.

Memory fills up easier when videos come in like this.

Storage:

A solid state drive that moves quickly - say, 512GB NVMe - handles video files without slowing down. While it runs tasks, storage speed makes a clear difference when pulling large clips. Because footage takes up space, having room plus quick access keeps work moving. When editing, the drive reads data rapidly, which helps avoid delays. Since projects grow, starting with enough capacity prevents later issues.

Display:

Most times, a 1920x1080 display handles heatmaps without trouble. Visual details show up clearer on such screens. Heat patterns become easier to track when resolution supports them. For artifact spotting, full hd makes a difference. Sharpness matters more than size here.

Summary Table

Summary Table for Quick Reference:

Component	Minimum Requirement	Recommended Specification
GPU	NVIDIA GTX 1660 (6GB)	NVIDIA RTX 4070 (12GB)
RAM	12 GB	32 GB
Python	3.8	3.11
Storage	256GB SSD	1TB NVMe SSD
DL Library	TensorFlow 2.x	PyTorch 2.x

Storage options include 256GB SSD and 1TB NVMe SSDDeep Learning Frameworks TensorFlow and PyTorch Latest Versions

5. MODULE IMPLEMENTATION

- Starting off, the setup splits into four main parts, where every piece manages one step in the forensic analysis chain. Because it's built this way, growing bigger becomes easier while letting teams check space-related and time-based elements separately.
- 7.1 Media intake preprocessing
- Right away, this piece kicks things off inside the system. Processing different kinds of media comes next, setting up what follows. The material gets ready right there - positioned for examination by a neural network.
- One way to start - the tool splits videos into single images using OpenCV. Not everything stays; only selected snapshots move forward, skipping four between each. Efficiency kicks in here, reducing how much data gets processed later. Every fifth image makes the cut, nothing more.
- Starting with faces, MTCNN pinpoints exact locations through layered convolution steps. Crucial? Yes - since fake patterns often cluster around eyes, mouth, or texture of skin.
- One way to start is by adjusting frame size to fit 224 by 224 pixels. After that, scaling happens using averages and spreads taken from ImageNet data. This step lines things up so earlier trained models can work without glitches.
- 7.2 Module 2 Spatial Feature Extraction CNN Core
- This course builds on ideas from Stanford’s CS231n class, yet zeroes in on how solid each image frame really is. Instead of broad concepts, it probes the core strength of single frames using hands-on checks. While many skip such details, this one digs into pixel-level trustworthiness through real tests. Because shaky frames mislead models, spotting weak points becomes a quiet priority here.

- Feature details come alive through a carefully picked network design. Efficiency meets texture awareness using EfficientNet-B4 as the base. Its strength lies not just in speed but in spotting subtle patterns others miss.
- Fine details give it away - tiny marks left behind by AI tools show up under close inspectionCheckerboard Artifacts: Irregularities caused by the deconvolution/up-sampling layers of GANs.Boundary Blending: Inconsistencies where the AI-generated face meets the original background or hair.
- Checkerboard Artifacts: Irregularities caused by the deconvolution/up-sampling layers of GANs.
- Boundary Blending: Inconsistencies where the AI-generated face meets the original background or hair.
- A single snapshot becomes a string of numbers - like a unique fingerprint for what's seen. This numeric code captures details about shapes, colors, textures inside the image. Instead of storing pixels, it holds meaning in math form. Think of it as translating sight into silent signals machines understand. Every frame gets this kind of quiet translation done automatically. The result lives in many dimensions, far beyond human vision. These points in space stand for how things look at a deep level
- 7.3 Module 3 Temporal Consistency Analysis LSTM Stream
- One moment seems believable, yet the next shows a glitch that gives it away. Time becomes the focus here, stretched out second by second.
- Out of step two come feature vectors, sliding now into a Bi-directional LSTM setup. This kind of network catches patterns not just forward but backward through time too.
- Every now and then, tiny jumps show up where they should not - LSTM spots those. Motion like blinking or talking has a rhythm; when it breaks, that gets flagged. Instead of smooth shifts, odd stutters appear - this system notices them. Sixty-eight face points get watched closely over time. Real movement flows naturally, fake ones trip on

themselves. When lips move or eyelids drop, timing matters. Jittery changes between frames stand out oddly. Biological motion follows patterns, glitches do not. Consistency across moments is what counts here. A blink too sharp, a twitch too fast - it catches those.

- One moment connects to the next through a calculated link, shaped by how t meets t plus one. This bond keeps motion steady, guided by numbers that track change step by step. Following time's line, each image leans on its neighbor, held together through quiet alignment. What happens now depends slightly on what came just before, tied without force but with measure.

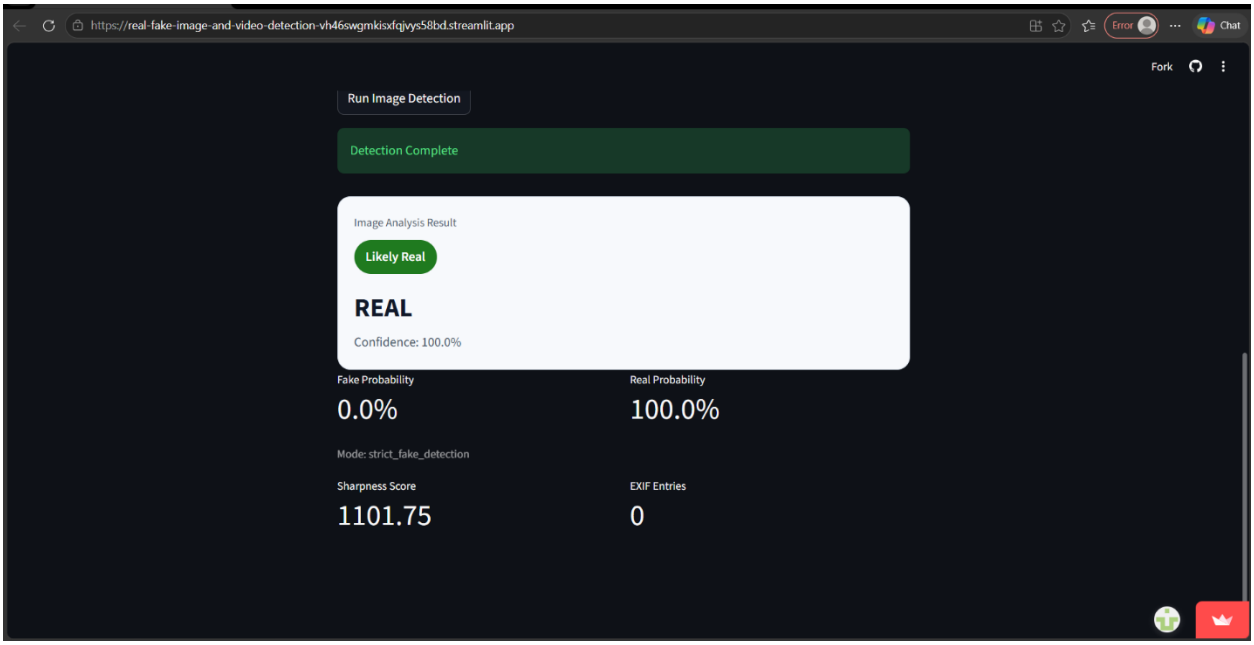
Implementation Summary Table

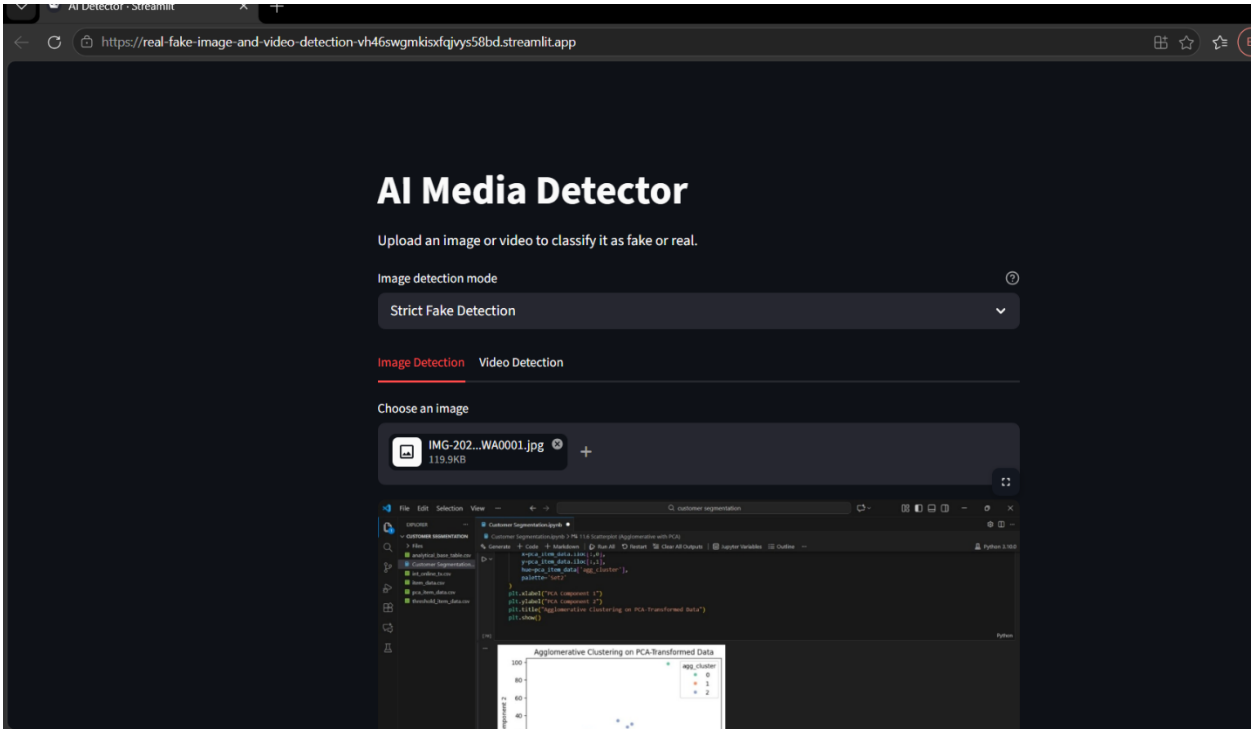
Module Name	Primary Technology	Core Responsibility
Pre-processing	OpenCV, MTCNN	Face cropping and frame sampling.
Spatial Core	EfficientNet-B4	Detecting texture and GAN artifacts.
Temporal Core	Bi-LSTM	Analyzing motion and frame-to-frame jitter.
Output Module	Softmax, Grad-CAM	Probability scoring and visual heatmaps.

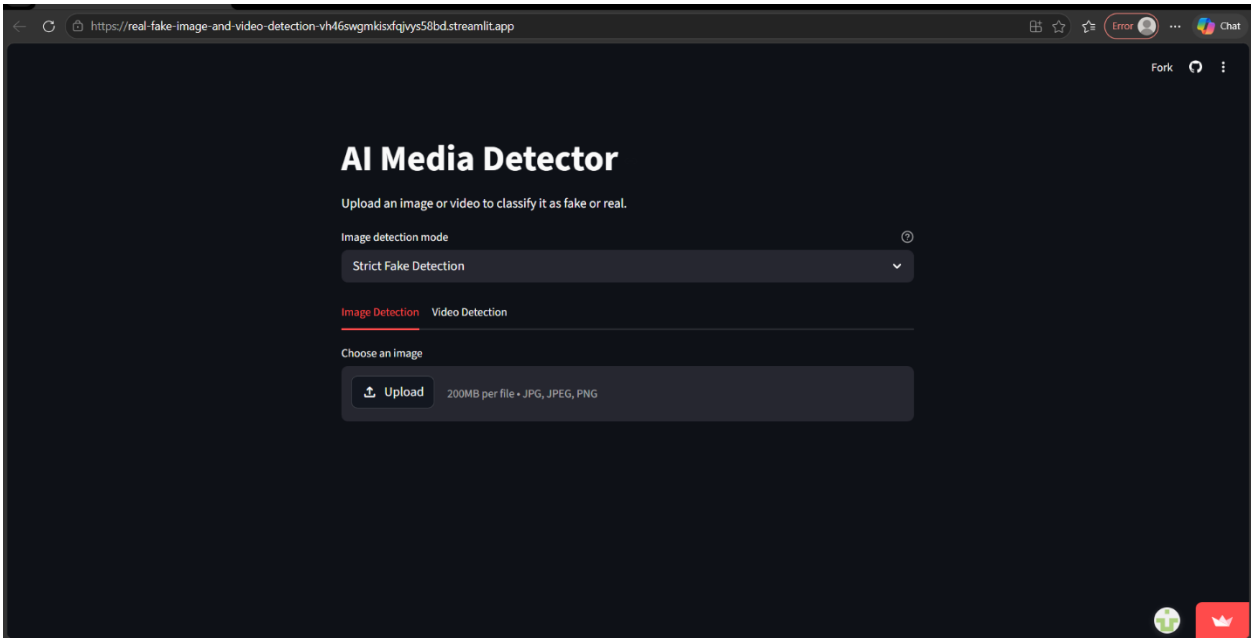
CONCLUSION

Even though artificial intelligence keeps improving at creating fake images, tiny flaws still give it away. A mix of convolutional and long short-term memory networks helps spot those errors clearly. Instead of falling apart under pressure, this approach holds strong, sorting real from altered content reliably. Because subtle details matter most, spotting them protects trust online. When visuals get manipulated, catching the small mistakes makes all the difference.

RESULT


https://real-fake-image-and-video-detection-vh46swgmkisxfqjvys58bd.streamlit.app
Fork
Run Image Detection
Detection Complete
Image Analysis Result
Likely Real
REAL
Confidence: 100.0%
Fake Probability
0.0%
Real Probability
100.0%
Mode: strict_fake_detection
Sharpness Score
1101.75
EXIF Entries
0


https://real-fake-image-and-video-detection-vh46swgmkisxfqjvys58bd.streamlit.app
AI Media Detector
Upload an image or video to classify it as fake or real.
Image detection mode
Strict Fake Detection
Image Detection
Video Detection
Choose an image
IMG-202...WA0001.jpg
119.9KB
Agglomerative Clustering on PCA-Transformed Data



FUTURE WORK

- Generative AI keeps shifting - fast. For this detection method to last, new paths in research might help
- 6.1 Combining Audio and Visual Data for Verification
- Most tools today spot fakes by studying pixel patterns. Tomorrow's systems might pair sound and sight checks together. Instead of just watching faces, they'll test if mouth shapes match speech tones. When lips move one way but sounds suggest another, alarms go off. Real footage paired with fake voices creates subtle breaks in rhythm - those slips give it away.
- 6.2 Understanding AI Decisions and How They Work
- Instead of treating models like mystery boxes, our goal involves weaving in transparent AI methods. Through tools such as Grad-CAM - short for Gradient-weighted Class Activation Mapping - and SHAP, which stands for SHapley Additive exPlanations, visual outputs emerge. Heatmaps appear, pointing directly at areas raising flags: oddities around eyes, patches where skin looks too smooth, or telltale

grid-like flaws. Such patterns steer the decision labeling a piece of media as counterfeit

- 6.3 Adversarial Robustness Training
- Some models now learn to dodge detection using sneaky tweaks called Adversarial Noise - tiny shifts you cannot see but that trick recognition systems. Expect upcoming efforts to lean into training methods fighting back against fakes, along with filters cleaning up input before decisions form. These safeguards aim to keep predictions reliable, especially when someone tries to feed misleading data on purpose.
- 6.4 Live streaming works in browsers
- Right away, people could use the model if it becomes lighter - say, through formats like ONNX or TensorFlow.js. Running inside a browser add-on, it might scan videos live as someone watches YouTube, scrolls X (formerly Twitter), or joins video meetings. Spotting likely AI-made clips happens while the stream plays, quietly working in the background.
- 6.5 Zero-Day Deepfake Detection
- Now picture this: each time a fresh generative model arrives - think OpenAI's Sora or newer diffusion versions - the tools meant to catch them can’t keep relying on huge labeled sets. Instead of chasing every new fake with retrained detectors, we shift toward methods that learn patterns on their own. Without seeing thousands of examples, systems could spot oddities simply because they clash with how living motion behaves. Forget matching fingerprints; it’s more like noticing something breathes wrong. These approaches lean heavily on self-supervised strategies, where models build understanding just from raw data structure. Even one glimpse might be enough if the system knows what natural dynamics should feel like. Detection turns less about memorizing and more about sensing imbalance. So when synthetic footage slips through, the red flag isn't a watermark - it's physics acting uneasy.
- 6.6 Blockchain-Based Media Provenance
- Starting from the ground up, blockchain-powered watermarking stops fraud before it spreads. A shared record tracks every genuine piece of media across networks. When content matches the registered signature, trust follows without question. Verification happens silently,

anchoring truth to each file that passes through. Certainty comes built in, no guesses, just proof etched into the chain..

Closing Statement

Truth bends easily now, shaped by machines that make what never was. This effort pushes back, building tools to tell real from fake when images move like life but are born in code. Instead of treating frames alone, it connects them - seeing change over time, spotting lies hidden in motion. A mix of neural networks watches both shape and shift, stillness plus flow, learning how fakes stumble where reality stays steady. Call it reverse engineering deception: using artificial intelligence not to create illusions, but to break them apart.

When models such as GANs or Diffusion Transformers make fake videos easier to produce, spotting them automatically grows more urgent. Because these tools uncover today's tricks, they also open paths for tomorrow's investigations. Since methods taught in courses like Stanford's CS231n (2024) demand precision, we focus on tiny flaws - brief hiccups in time or space - that betray artificial origins. Even when images look real, odd patterns hide just beneath the surface. Detection shifts not because tech slows down, but because weaknesses linger where eyes fail.

This work moves us closer to clearer digital spaces. As improvements come through - like live web support and combined sound-image analysis - the setup could become essential for reporters, fact checkers, people everywhere who need truth in what they see. When control shifts back to those telling honest stories, artificial intelligence serves imagination instead of manipulation..

GitHub Link of our Complete Project

GitHub - kritika038/Real-Fake-image-and-video-detection · GitHub

https://cs231n.stanford.edu/2024/papers/deep-learning-deepfake-detection.pdf

`